

Mt

11 November 2023 Saturday

01:56

Q2: You are to sort n employees in a company into k **equal-sized groups** by income (assume that no two employees have the same income, n is a multiple of k , and k is a power of 2 for simplicity). Denote these groups as $G_1, G_2, \dots, G_k$. The groups must have the property that $\forall i < j$, every person in G_i has a lower income than every person in G_j . The ordering within any particular group is of **no** significance. Give an $O(n \cdot \lg k)$ -time algorithm to produce k such groups to deserve full credit. An expected $O(n \cdot k)$ -time algorithm will deserve 12 pts.

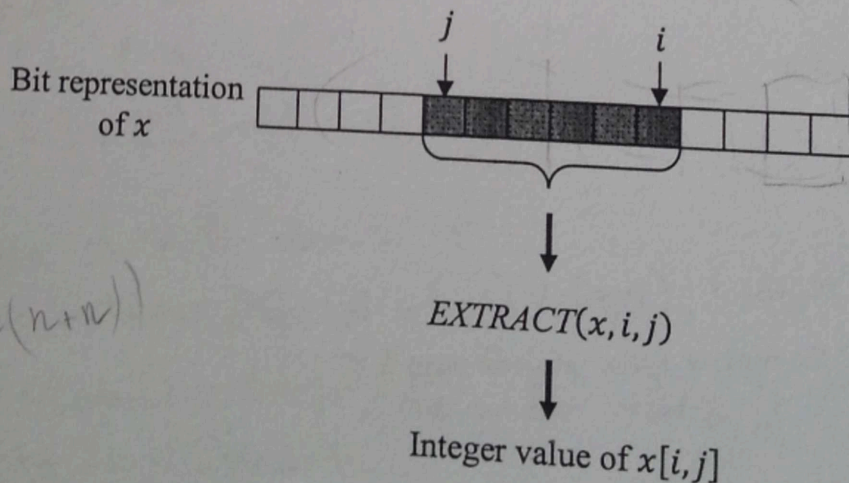
Q3: Given a set of n numbers, we wish to find and return the largest $i \leq n$ numbers in **sorted** order using a comparison-based algorithm.

- A couple of algorithms to address the above problem are given below in i. and ii. Analyze the worst-case running times of these algorithms in terms of n and i , assuming they are implemented so as to yield **best asymptotic worst-case running times**. Mention any critical details.
 - Sort the numbers, and list the i largest.
 - Build a max-heap from the numbers, and call `HEAP-EXTRACT-MAX` i times.
- Describe another algorithm (in pseudocode) with a **potentially better asymptotic worst-case running time** than those in a., clearly stating under what condition this applies. Provide a complexity analysis.

✓ b. (30 pts.) Given two positive integers $1 < k < l < n$, describe an **expected linear-time, in-place** algorithm which permutes the numbers in A , such that the numbers whose ranks are between k and l are placed in between the k^{th} and l^{th} locations of the A -array. Note that order of the numbers between the k^{th} and l^{th} locations is not important. For example, for $A = \{5, 8, 1, 6, 9, 2, 3, 7, 4\}$ and $k = 4$ and $l = 7$, permuted array may look like $A = \{*, *, *, 7, 5, 4, 6, *\}$ or $A = \{*, *, *, 6, 4, 5, 7, *\}$ etc.

3. (35 pts)
Given n elements stored in k sorted arrays $A_1[1 \dots n_1], A_2[1 \dots n_2], \dots, A_k[1 \dots n_k]$, where $n_1 + n_2 + \dots + n_k = n$, describe an $O(k + m \lg k)$ -time algorithm to find largest m of the n elements stored in $A_1, A_2, \dots, A_k$. You can use the notation $A_i[j]$ to access the j^{th} element of array A_i in constant time.
Hint: Consider maintaining a heap of size k .

2. You are given an array $A[1..n]$ of n positive integers in the range 0 to $n^4 - 1$. Describe a linear-time algorithm to sort the integers in A . You can assume you have a **constant-time** function $EXTRACT(x, i, j)$, which, given a positive integer x and two bit positions i and j , returns the integer value of the bits between i^{th} and j^{th} position of x (both inclusive), where $i \leq j$.



2) We can use Radix Sort-like algorithm.

$T_k = \Theta(d(n+k))$ We need to make d constant and $k = o(n)$

$\Rightarrow$ Numbers in range 0 to n^4 has $\log n^4$ bits = $4 \log n$

We can pretend that numbers in the array has 4 digits. And in that case each digit has $\log n$ bits, which means that digits range from 0 to $n \Rightarrow T(n) = \Theta(4(n+n)) = \Theta(n)$. For simplicity it is assumed that n is a power of 2 .

$SORT(A)$

$no_of_bits = \log(A.length)$

for $i = 1$ to 4

$DIGIT_SORT(A, A.length, i)$ // $A.length$ is n which is max digit
// for counting sort

$DIGIT_SORT(A, k, digit)$ // a counting-sort-like algorithm

$n = A.length$

for $i = 0$ to k

$C[i] = 0$

for $j = 1$ to $A.length$

(50)

$C[\text{EXTRACT}(A[j], ((\text{digit}-1) * \log n), (\text{digit} * \log n - 1))]++$ //incremented
 //instead of the number itself, use only a part of it as a numeric
 //value.

for $i = 1$ to k

$C[i] = C[i] + C[i-1]$

for $j = A.\text{length}$ down to 1

$\text{idx} = \text{EXTRACT}(A[j], (\text{digit}-1) * \log n, (\text{digit} * \log n - 1))$

$B[C[\text{idx}]] = A[j]$ //even though only a part is used for counting, copy
 //the number itself

$C[\text{idx}] = C[\text{idx}] - 1$

for $j = 1$ to $A.\text{length}$ //copy values back to the original array.

$A[j] = B[j]$

Counting sort is modified in a way that it uses only a part of the numbers to sort them. As this sort is stable, it is used by radix sort.

3. Derive the expression for the recurrence that describes the expected running time of the Randomized Quicksort algorithm as detailed in lecture notes. Recall that Randomized Quicksort algorithm uses Hoare's Partitioning algorithm in the divide step. The derived recurrence should be in simplified form as much as possible.

that, given n distinct numbers stored in array $A[1 \dots n]$,

exists no two numbers with the same close

5. Let u and v be two n -bit binary numbers, where for simplicity n is a power of 2. The traditional multiplication requires $\Theta(n^2)$ bit operations, where each operation is either the multiplication or addition of two 1-bit values. A straightforward divide-and-conquer approach splits the numbers into two equal sized parts, computing the product as:

$$u \times v = (a \times 2^{n/2} + b) \times (c \times 2^{n/2} + d) = a \times c \times 2^n + (a \times d + b \times c) \times 2^{n/2} + b \times d$$

However, this straightforward approach leads to a $\Theta(n^2)$ -time algorithm. Propose and describe (in pseudocode) a new divide-and-conquer based algorithm which solves the problem asymptotically faster than $\Theta(n^2)$. Indicate the running times (in number of bit operations) of the three phases of your divide-and-conquer approach. Write the recurrence describing the running time of your algorithm. Determine the running time of your algorithm by solving the recurrence.

Q1a. (30 pts)

For a given value n , the sorted (in increasing order) input array A stores all numbers $1, 2, 3 \dots, n$ except one number (The length of array A is $n - 1$).

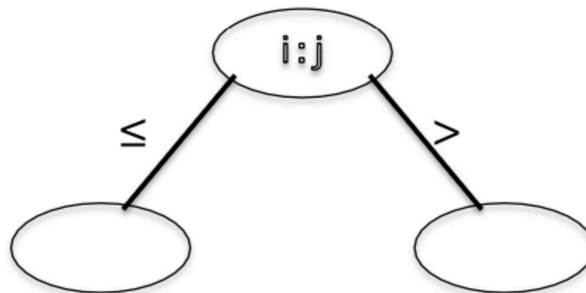
- Describe a divide-and-conquer-based algorithm (in **pseudocode**) that finds and returns the missing number. Your algorithm should run in $O(n)$ time (It should be asymptotically faster than linear time).
- Give the recurrence that describes the running time of your algorithm and then solve the recurrence to show the running time.

Examples

- For the array $[1, 2, 4, 5]$, the answer is 3.
- For the array $[1, 2, 3, 4]$, the answer is 5.
- For the array $[2, 3, 4, 5]$, the answer is 1.

Q1 (40 pts)

Draw the **decision tree** for BUILD-HEAP on the input array $A = [a_1, a_2, a_3, a_4]$. Hint: The decision tree does not necessarily contain all permutations of $[a_1, a_2, a_3, a_4]$ as leaf nodes. Use the following notation when drawing the non-leaf nodes in decision tree;



In this notation, i and j respectively denote a_i and a_j . You should use the following notation for showing the leaf nodes; $[a_{i_1}, a_{i_2}, a_{i_3}, a_{i_4}]$.

The following pseudocodes are taken from lecture slides;

BUILD-HEAP (A, n)

for $i = \lfloor n/2 \rfloor$ downto 1 do
HEAPIFY(A, i, n)

HEAPIFY (A, i, n)

largest $\leftarrow i$

if $2i \leq n$ and $A[2i] > A[i]$
then largest $\leftarrow 2i$

if $2i+1 \leq n$ and $A[2i+1] > A[\text{largest}]$
then largest $\leftarrow 2i+1$

if largest $\neq i$ then

exchange $A[i] \leftrightarrow A[\text{largest}]$
HEAPIFY($A, \text{largest}, n$)

Q2. (40 pts)

You are given an array A of n distinct numbers and two other numbers p_1 and p_2 such that $p_1 < p_2$, $p_1 \notin A$ and $p_2 \notin A$.

For each element $x \in A$,

- the distance $d_1(x)$ of x to p_1 is defined as the number of elements in A that are numerically between p_1 and x .
- the distance $d_2(x)$ of x to p_2 is defined as the number of elements in A that are numerically between p_2 and x .
- the pair-distance of x to (p_1, p_2) is defined as $\min(d_1(x), d_2(x))$.

Give a worst-case linear time algorithm in **pseudocode** that finds an element in A that has the maximum pair-distance to (p_1, p_2) .

Note that the solution is either the minimum element of A , the maximum element of A or a number x of A that has equal distance to p_1 and p_2 .

Example

$A = [13, 15, 19, 17, 24, 76, 42, 1, 4]$, $p_1 = 10$ and $p_2 = 30$

	1	4	13	15	17	19	24	42	76
$d_1(x)$	1	0	0	1	2	3	4	5	6
$d_2(x)$	6	5	4	3	2	1	0	0	1
pair-distance	1	0	0	1	2	1	0	0	1

The answer is 17. It has the maximum pair-distance.

$d_1(17) = |\{13, 15\}| = 2$ (13 and 15 are the only numbers between $p_1 = 10$ and $x = 17$)

$d_2(17) = |\{19, 24\}| = 2$ (19 and 24 are the only numbers between $x = 17$ and $p_2 = 30$)

Q1. (50 pts)

Suppose you are given an array $A[1..n]$ of distinct and sorted integers that has been circularly shifted k positions to the right. For example, $[35, 42, 5, 15, 27, 29]$ is a sorted array that has been circularly shifted $k = 2$ positions, while $[27, 29, 35, 42, 5, 15]$ has been shifted $k = 4$ positions. Describe an algorithm that finds the shift amount (k) in $o(n)$ (little-oh) time. Make sure to analyze the running time of your algorithm.

Chat

Q1. (50 pts)

You are given an $m \times n$ matrix A such that the entries of each row are in non-decreasing sorted

order from left to right and the entries of each column are in non-decreasing sorted order from top to bottom. Some elements of the matrix may be $+\infty$ to indicate nonexistent elements. Below is an example of such a matrix:

1	4	5	7
3	6	8	11
5	9	16	$+\infty$
6	10	$+\infty$	$+\infty$
8	12	$+\infty$	$+\infty$

Give an $\mathcal{O}(m+n)$ -time algorithm that performs EXTRACT-MIN operation on A . Your algorithm should maintain the structure of the matrix after performing the operation, i.e. the rows and the columns of the matrix should satisfy the property of being sorted as mentioned above.

- (a) (5 pts) First the idea,
- (b) (40 pts) Then the algorithm in pseudocode,
- (c) (5 pts) And analyze the run time of your algorithm and indicate why it works in $\mathcal{O}(m+n)$.

Q2. (50 pts)

- (a) (25 pts) Let A be an array of n (not necessarily distinct) integers. Describe a worst-case $\mathcal{O}(n)$ -time algorithm in pseudocode to test whether any number occurs more than $\lceil \frac{n}{2} \rceil$ times in A . Return true or false. (Do not try to use hashing.).
- (b) (25 pts) Suppose that you need to sort 1000 numbers between 0 and $10^6 - 1$ (i.e. the keys are 6 digit numbers). You have decided to use radix sort that uses counting sort as the auxiliary stable sort but first you need to decide how many digits to group for each radix sort digit. Which is the best among having 1 digit per radix sort digit, 3 digits per radix sort digit or 6 digits per radix sort digit? You are provided with a counting sort procedure with exact run time of $5n + 4k$ steps, where n is the number of elements in input and k is the range of input. First fill the below table accordingly and then decide which grouping is better (ignore time spent outside counting sort and consider only # of steps during counting sort).

digits / radix digit	k	# of radix digits	# of steps to run
1			
3			
6			

Algorithm 1 TEST(A, n)

```

1:  $m \leftarrow \text{MM} - \text{SELECT}(A, n, \lceil \frac{n}{2} \rceil)$ 
2:  $\text{freq} \leftarrow 0$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:   if  $A[i] = m$  then
5:      $\text{freq} \leftarrow \text{freq} + 1$ 
6: if  $\text{freq} > \frac{n}{2}$  then
7:   return true
8: else
9:   return false

```

- (b) (25 pts) Suppose that you need to sort 1000 numbers between 0 and $10^6 - 1$ (i.e. the keys are 6 digit

numbers). You have decided to use radix sort that uses counting sort as the auxiliary stable sort but first you need to decide how many digits to group for each radix sort digit. Which is the best among having 1 digit per radix sort digit, 3 digits per radix sort digit or 6 digits per radix sort digit? You are provided with a counting sort procedure with exact run time of $5n + 4k$ steps, where n is the number of elements in input and k is the range of input. First fill the below table accordingly and then decide which grouping is better (ignore time spent outside counting sort and consider only # of steps during counting sort).

digits / radix digit	k	# of radix digits	# of steps to run
1	10^1	6	$6(5000 + 40) = 30240$
3	10^3	2	$2(5000 + 4000) = 18000$
6	10^6	1	$5000 + 4 \times 10^6 = 4005000$

According to the results, best choice is 3 digits per radix sort digit.

Q3: Given an n -element array A with distinct integer keys, you are asked to output the k elements with smallest keys that are greater than the median key in sorted order (Example: $A = [5, 2, 4, 3, 7, 1, 6]$ and $k = 2$ returns $[5, 6]$ since 4 is the median key and the two elements with smallest keys greater than the median are these):

- a. Provide an $O(n)$ -time algorithm for the case when k is constant?

We can first find the median m and $n/2 + k^{\text{th}}$ smallest mk in A (using MM-SELECT) and store all values in $[m, mk)$ between in a new array B . Then we can apply any $O(n \log n)$ -time sorting algorithm on these elements before outputting. The whole process should take $\Theta(n) + \Theta(n) + \Theta(n) + O(k \log k) = \Theta(n)$ time since k is a constant.

```
k-Smallest-After-Median(A, n, k)
    median = MM-Select(A, n, n/2)
    medianPlusK = MM-Select(A, n, n/2+k)
    B = Find-In-Range(A, median, medianPlusK) //returns x in [median, medianPlusK)
    Merge-Sort(B, 1, k)
    return B
```

Alternatively find the median and build a min-heap from keys greater than or equal to the median. Then perform k extracts.

- b. Provide an $O(n)$ -time algorithm for the case when $k = n^{2/3}$?

When $k = n^{2/3}$, the whole process should take $\Theta(n) + \Theta(n) + \Theta(n) + O(n^{2/3} \log n^{2/3}) = \Theta(n)$ time.

- c. Assuming $k = n^c$, where c is a constant, what is the largest value of c for which your algorithm for part b is $O(n)$?

So long as $O(n^c \log n^c) = O(n^c \log n) = O(n)$, we should be fine. This is satisfied for any $c < 1$.

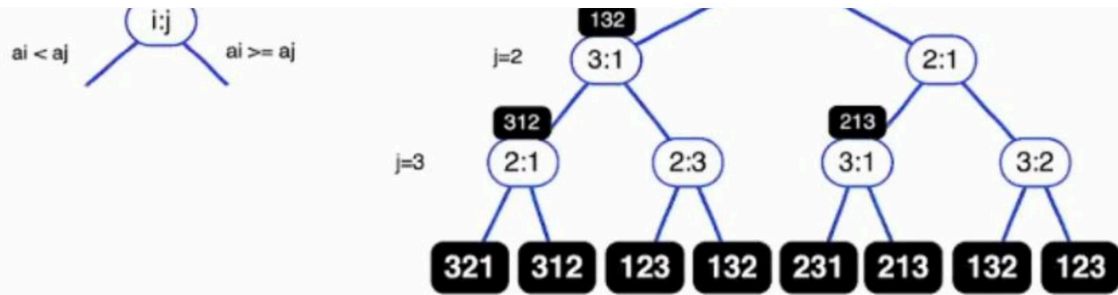
You may provide a single algorithm for parts a and b or different ones. In any case, first provide the idea behind your algorithm(s), followed by pseudocode, and finally a run-time analysis of your algorithm(s).

Q4:

- a. (13pts) Draw the decision tree for the bubble-sort algorithm operating on an array $A = [a_1, a_2, a_3]$ of three numbers. For the sake of simplicity of your drawing, you can show element comparisons ($a_i : a_j$) as $(i : j)$.

```
BUBBLE-SORT(A, n=A.length)
for i=1 to n-1 do
    for j=n downto i+1 do
        if A[j] < A[j-1]
            exchange A[j] with A[j-1]
```





- b. (12pts) You are given an array $A[1..n]$ of n positive integers in the range 0 to $n^3 - 1$. Describe a linear-time algorithm to sort the integers in A . Make sure to argue why your algorithm's worst case running time is linear.

Radix Sort takes $O(d * (n + b))$ time where b is the base for representing numbers. Since $n^3 - 1$ is the maximum possible value, the value of d would be $O(\log_b n)$. So if b is chosen to be n , then $d = O(\log_b n) = O(1)$. Hence, $O(d * (n + b)) = O(1 * (n + n)) = O(n)$.

Q2: [30min] (20pts) Let $X[1..n]$ and $Y[1..n]$ be two arrays, each containing n numbers already in sorted (ascending) order. Consider the problem of finding the median of all $2n$ elements in arrays X and Y . Remember to take the average of the two numbers in the middle when all $2n$ numbers (average of n th and $n+1$ st among $2n$ numbers) are sorted as the median. Describe an algorithm to find the median of such two sorted arrays in $O(\log n)$ time. Asymptotically slower algorithms will get partial credit (up to 8pts).

Use a binary search like procedure simultaneously on both arrays:

```
median-of-2-arrays(X, p1, r1, Y, p2, r2) // initial call with (X, 1, n, Y, 1, n)
    n = (r1-p1)+1
    if n < 1 return -1 // invalid input
    if n = 1 return (X[p1] + Y[p2]) / 2 // when only 1 left from each array
    m1 = median(X, p1, r1)
    m2 = median(Y, p2, r2)
    if m1 == m2 return m1
    if m1 < m2 // ignore first half of X and second half of Y
        return median-of-2-arrays(X, p1+n/2-1, r1, Y, p2, p2+n/2)
    else
        return median-of-2-arrays(X, p1, p1+n/2, Y, p2+n/2-1, r2);

median(A, p, r) // returns median of range in array
    n = (p-r)+1
    q = (p+r)/2
    if n % 2
        return (A[q]+A[q+1])/2
    else
        return A[q]
```

Longest Palindromic Subsequence: D-n-C solution

```
int findLPSLength(X, p, r)
    if p > r return 0
    // a sequence with single element is a palindrome of length 1
    if p == r return 1
    // beginning and end elements are same
    if X[p] == X[r]
        return 2 + findLPSLength(X, p+1, r-1)
    // ignore one non-matching element either from beginning or from end
    l1 = findLPSLength(X, p+1, r)
    l2 = findLPSLength(X, p, r-1)
    return max(l1, l2)
```

Q2: You are given a sorted array A of n positive integers. Describe an algorithm that, given a number x , determines whether or not there exist two numbers in A whose sum is exactly equal to x . Your algorithm should return the pair of integers whose sum is equal to x if such a pair exists, and should return NIL otherwise. An $O(n)$ -time algorithm will deserve full credit, whereas an asymptotically worse one will get partial credit up to 8pts.

a. First the idea,

Since the array is sorted, we can have start from the two ends and try to find the right pair that sums to the given number x in linear time.

b. Then the algorithm in pseudocode,

```
sum-of-two(A, n, x)
  i=1
  j=n
  while (i<j)
    if (A[i]+A[j] = x) // bingo
      return <A[i],A[j]>
    else if (A[i]+A[j] < x) // too small
      i++
    else // too big
      j--
  return NIL
```

c. And the run time analysis.

This is clearly linear in the number of integers in the input A .

3. (15pts) Find a **tight** asymptotic upper bound for the running time of an algorithm with the provided recurrence using the Substitution method:

$$T(n) = 8T(n/2) + n^2, n \geq 1$$

$$T(1) = O(1)$$

Guess $T(n) = O(n^3)$

Inductive hypotheses: assume $T(k) \leq ck^3, k < n$

Inductive step: need to show that $T(n) \leq cn^3$

$T(n/2) \leq c(n/2)^3$ by the inductive hypotheses $k = n/2$

$$8T(n/2) + n^2 \leq 8c(n/2)^3 + n^2$$

$$T(n) \leq cn^3 + n^2 \text{ stuck!}$$

Tighten guess $T(n) \leq cn^3 - bn^2$

Inductive hypotheses: assume $T(k) \leq ck^3 - bk^2, k < n$

Inductive step: need to show that $T(n) \leq cn^3 - bn^2$

$T(n/2) \leq c(n/2)^3 - b(n/2)^2$ by the inductive hypotheses $k = n/2$

$$8T(n/2) + n^2 \leq 8[c(n/2)^3 - b(n/2)^2] + n^2$$

$$T(n) \leq cn^3 - 2bn^2 + n^2 = cn^3 - bn^2 - (bn^2 - n^2)$$

$$T(n) \leq cn^3 - bn^2 \text{ for } b \geq 1 \text{ and } n \geq n_0$$

To prove that this in fact is a tight bound, we will also show the same function to be lower bound.

Guess $T(n) = \Omega(n^3)$

Inductive hypotheses: assume $T(k) \geq ck^3, k < n$

Inductive step: need to show that $T(n) \geq cn^3$

$T(n/2) \geq c(n/2)^3$ by the inductive hypotheses $k = n/2$

$$8T(n/2) + n^2 \geq 8c(n/2)^3 + n^2$$

$$T(n) \geq cn^3 + n^2 \geq cn^3$$

First the idea or proof that it cannot be done in specified run time:

The idea is to build a min heap out of all n elements and find its $n^{2/3}$ smallest ones by repeatedly extracting the minimum element.

Now the algorithm:

```
Find-Smallest-2-3rd-Sorted( $A, p, r$ )
   $n \leftarrow r - p + 1$ 
   $k \leftarrow \lfloor n^{2/3} \rfloor$ 
  Build-Min-Heap( $A, n$ )
  for  $i \leftarrow 1$  to  $k$  do
    output Heap-Extract-Min( $A, n$ )
     $n \leftarrow n - 1$ 
```

And the run-time analysis:

Since extracting max is an operation of $O(\log n)$ run time complexity called $\lfloor n^{2/3} \rfloor$ times, overall, the given algorithm runs in $O(\lfloor n^{2/3} \rfloor \log n)$ time since $\log n = o(n^{1/3})$.